FP2D_QLRF_NL

User Manual

2D Fokker–Planck solver for ion velocity distribution functions in cylindrical velocity space

F. Louche
LPP-ERM/KMS

Version 3.1 – 21 October 2026

Contents

1 Introduction **3**

2 Physical Model **3**
2.1 The Fokker–Planck Equation . . . 3
2.2 Coulomb Collision Operator . . . 3
2.2.1 Coulomb Logarithm . . . 4
2.3 Quasi-Linear RF Operator . . . 4
2.4 Self-Collision Operator . . . 4
2.5 Beam Source and Particle Losses . . . 5

3 Numerical Methods **5**
3.1 Velocity-Space Grid . . . 5
3.2 Finite-Difference Discretization . . . 6
3.2.1 Boundary Conditions . . . 6
3.3 Advection Stabilization at High $v_\perp$ (`i_upwind`) . . . 6
3.4 Sparse Matrix Storage . . . 7
3.5 Direct Solver: Intel MKL PARDISO . . . 7

4 Algorithms **7**
4.1 Steady-State Solver . . . 7
4.2 Time-Dependent Solver – Linear Problem (`timefp_7pt`) . . . 8
4.2.1 Time Discretization . . . 8
4.2.2 Algorithm . . . 8
4.3 Time-Dependent Solver – Nonlinear Self-Collisions (`timefp_7pt_nl`) . . . 8
4.3.1 Phi-Distance Kernel Precomputation . . . 8
4.3.2 Time-Stepping Scheme . . . 9
4.3.3 Algorithm . . . 9
4.4 Multi-Phase Time-Stepping . . . 9
4.5 Initial Condition for Time-Dependent Runs . . . 10
4.6 Steady-State Convergence Detection . . . 10
4.7 Diagnostics and Output . . . 11

5 Namelist Reference **12**
5.1 General and Output Control . . . 12
5.2 Velocity-Space Grid . . . 12

5.3 Plasma Parameters . . . 13
5.4 Beam Source . . . 13
5.5 RF Heating . . . 14
5.6 Time Integration and Solver Control . . . 14
5.7 Steady-State Convergence Detection . . . 16

6 Output Files **17**
6.1 Final Distribution . . . 17
6.2 VDF Snapshots . . . 17
6.3 1D Cuts of the VDF . . . 18
6.4 Time Traces . . . 18
6.5 Other Files . . . 19
6.6 End-of-Run Balance Diagnostics (stdout) . . . 19

7 Example Namelist **19**

8 Compilation and Execution **20**
8.1 Build . . . 20
8.2 Running . . . 21
8.3 Plotting . . . 21

1 Introduction

`FP2D_QLRF_NL` solves the two-dimensional Fokker–Planck (FP) equation for the velocity distribution function (VDF) $f(v_\perp, v_\parallel, t)$ of a minority ion species in a magnetised plasma. The two velocity coordinates are the speed perpendicular to the magnetic field $v_\perp \geq 0$ and the speed parallel to the field $v_\parallel \in (-\infty, +\infty)$. Azimuthal symmetry about the magnetic field direction is assumed throughout.

The code accounts for three physical operators:

1. **Coulomb collisions** with one or more Maxwellian background species (electrons and bulk ions).
2. **Quasi-linear radio-frequency (RF) heating** via the quasi-linear diffusion operator, evaluated using complex Bessel functions.
3. **Self-collisions** (collisions of minority ions with themselves), with five levels of approximation selectable at runtime.

An optional NBI **beam source** term and an associated particle **loss term** are also included.

The code can compute either a *steady-state* solution (direct solve) or a *time-dependent* evolution (implicit time integration). Spatial discretization uses a 7-point Fornberg finite-difference stencil. The resulting sparse linear systems are solved with Intel MKL PARDISO.

2 Physical Model

2.1 The Fokker–Planck Equation

The VDF $f = f(v_\perp, v_\parallel)$ evolves according to

$$\frac{\partial f}{\partial t} = A\, f + B\, \frac{\partial f}{\partial v_\perp} + C\, \frac{\partial f}{\partial v_\parallel} + D\, \frac{\partial^2 f}{\partial v_\perp^2} + E\, \frac{\partial^2 f}{\partial v_\perp\, \partial v_\parallel} + F\, \frac{\partial^2 f}{\partial v_\parallel^2} + S(v_\perp, v_\parallel), \tag{1}$$

where the six coefficient arrays A–F (called `all00`, `all10`, `all01`, `all20`, `all11`, `all02` in the code) are assembled as the sum of three independently computed contributions:

$$X = X_{\mathrm{coll}} + X_{\mathrm{RF}} + X_{\mathrm{SC}}, \qquad X \in \{A, B, C, D, E, F\}, \tag{2}$$

for Coulomb collisions, RF heating, and self-collisions, respectively. The source term S collects the NBI beam injection and the particle loss.

In steady-state mode ($\partial f/\partial t = 0$) the right-hand side of (1) is set to zero and the resulting linear system is solved directly.

2.2 Coulomb Collision Operator

The Coulomb collision operator is the linearized Fokker–Planck operator describing the interaction of the minority ion species a (charge Z_a, mass m_a, thermal velocity v_a) with a Maxwellian background species b (charge Z_b, mass m_b, thermal velocity v_{Tb}, density n_b). The coefficient arrays (`colin00`, …, `colin02`) for each background species are computed by the subroutine `cblin` and summed over all `nbulk` background species.

The collision frequency scale factor is

$$\Gamma_b = \frac{n_b\, Z_a^2\, Z_b^2\, e^4\, \ln\Lambda_{ab}}{4\pi\, \varepsilon_0^2\, m_a^2}, \tag{3}$$

where $\ln\Lambda_{ab}$ is the Coulomb logarithm evaluated at each time step (see Section 2.2.1).

2.2.1 Coulomb Logarithm

Two distinct formulas are used:

Ion–electron:

$$\ln \Lambda_{ae} = 24 - \ln\left(\frac{n_e^{1/2}}{T_e}\right), \tag{4}$$

with T_e in eV and n_e in m^{-3}.

Ion–ion:

$$\ln \Lambda_{ab} = 23 - \ln\left[Z_a^2 Z_b^2 \, \frac{m_a + m_b}{m_a T_b + m_b T_a} \left(\frac{n_a Z_a^2}{T_a} + \frac{n_b Z_b^2}{T_b}\right)^{1/2}\right]. \tag{5}$$

In time-dependent runs the Coulomb logarithm is recomputed at each time step using the current effective temperature of the minority ion species.

2.3 Quasi-Linear RF Operator

When radio-frequency heating is active (`irf` $= -1$), the quasi-linear diffusion operator is added to the coefficient arrays. The RF operator describes the resonant power exchange between the minority ions and an electromagnetic wave characterised by:

- Wave frequency $\omega = 2\pi f$ (`frek` in Hz).
- Parallel wave vector $k_\parallel$ (`kpar` in rad/m).
- Perpendicular wave vector $k_\perp$ (complex, `kperp` in rad/m).
- Left- and right-hand polarisation components E_+, E_- (`eplus`, `emin`).
- Static magnetic field B_0 (`b0` in T).
- Harmonic number n (`nharm`).

The quasi-linear diffusion coefficients are computed in the subroutine `qlrfterm` using complex Bessel functions $J_n(k_\perp v_\perp/\omega_c)$ and their derivatives, evaluated at each grid point. The resonance condition is $\omega - n\,\omega_c - k_\parallel\, v_\parallel = 0$, broadened by a Gaussian of width `delta_RF` in $v_\parallel$.

2.4 Self-Collision Operator

Four options are available for the treatment of self-collisions, selected by the flag `isc`:

`isc = 0` – **No self-collisions.** The self-collision coefficients are zero. The FP equation is linear.

`isc = 1` – **Maxwellian background at fixed temperature.** The self-collision operator is computed using the Rosenbluth potentials for a Maxwellian distribution at the Stix effective temperature T_{eff} (computed once at the start of the simulation and kept constant). This adds a fixed linear operator to the coefficient arrays.

`isc = 2` – **Maxwellian background at varying effective temperature.** Same as `isc = 1`, but the background temperature and Coulomb logarithm are updated at each time step to track the evolving effective temperature $T_{\mathrm{eff}}(t) = (2T_\perp + T_\parallel)/3$. Requires a time-dependent run.

`isc = 3` – **Maxwellian background at the density-characteristic temperature.** Same as `isc = 2`, but the background temperature is the *density-characteristic* (cold-bulk) temperature $T_n(t)$ instead of the energy-weighted $T_{\rm eff}(t)$. T_n is obtained from a phase-space-weighted least-squares fit of $\ln f$ versus v^2 restricted to the dense thermal core — only cells with $f >$ `core_frac`$\cdot \max f$ enter the fit — which returns the temperature exactly for a Maxwellian. Being tied to the cold bulk rather than to the supra-thermal tail, this background better represents the population that dominates the collisional drag. The core fraction `core_frac` (default 3.8×10^{-3}) is tunable; the appropriate value depends on the shape of the distribution, so it may need re-calibrating against the fully nonlinear operator for a new scenario. Requires a time-dependent run.

`isc = -1` – **Fully nonlinear self-collisions.** The complete nonlinear Landau/Fokker–Planck self-collision operator

$$\left(\frac{\partial f}{\partial t}\right)_{\rm SC} = \Gamma_{aa}\,\frac{\partial}{\partial v_i}\left[H_{ij}[f]\,\frac{\partial f}{\partial v_j} - G_i[f]\,f\right], \tag{6}$$

is used, where H_{ij} and G_i are the Rosenbluth potentials built from the *current* distribution $f(t)$. The Rosenbluth potentials are evaluated via a precomputed phi-distance kernel `sum_phi` using Gauss–Legendre quadrature (subroutine `distance_v_gauss_legendre`). This kernel depends only on the velocity grid and is computed once and saved to disk (`sum_phi.dat`); it can be reloaded for subsequent runs on the same grid. The operator is rebuilt at every time step, making the problem nonlinear. Requires a time-dependent run.

2.5 Beam Source and Particle Losses

When `isource` $= -1$, a neutral-beam injection (NBI) source is added:

$$S(v_\perp, v_\parallel) = \frac{n_0}{\tau_s}\,\frac{\delta_G(v_\perp - v_\perp^{\rm beam})\,\delta_G(v_\parallel - v_\parallel^{\rm beam})}{\mathcal{N}}, \tag{7}$$

where $\delta_G(x) = e^{-(x/\sigma)^2}/(\sqrt{\pi}\,\sigma)$ is a Gaussian approximation to the Dirac delta, $v_\perp^{\rm beam} = v_{\rm beam}\sin\alpha$, $v_\parallel^{\rm beam} = v_{\rm beam}\cos\alpha$ with α the injection angle, and $\mathcal{N}$ is the normalization factor

$$\mathcal{N} = \sqrt{\pi}\,\sigma_\perp\,e^{-(v_\perp^{\rm beam}/\sigma_\perp)^2} + \pi\,v_\perp^{\rm beam}\,{\rm erfc}(-v_\perp^{\rm beam}/\sigma_\perp), \tag{8}$$

which reduces to $2\pi\,v_\perp^{\rm beam}$ for $v_\perp^{\rm beam} \gg \sigma_\perp$ and to $\sqrt{\pi}\,\sigma_\perp$ for a purely parallel beam ($v_\perp^{\rm beam} = 0$). The loss term is $-f/\tau_s$ (isotropic particle losses with slowing-down time τ_s).

3 Numerical Methods

3.1 Velocity-Space Grid

The computational domain is $[v_{\perp,\min}, v_{\perp,\max}] \times [v_{\parallel,\min}, v_{\parallel,\max}]$, discretized on a $N_\perp \times N_\parallel$ grid with 1-based indices (i, j) where $i \in [1, N_\perp]$ and $j \in [1, N_\parallel]$.

Three grid spacing modes are available, controlled by `ising`:

`ising = 0` Uniform spacing in both directions. $\Delta v_\perp = (v_{\perp,\max} - v_{\perp,\min})/(N_\perp - 1)$, $\Delta v_\parallel = (v_{\parallel,\max} - v_{\parallel,\min})/(N_\parallel - 1)$.

`ising = -1` Two-domain grid in $v_\perp$. A finer mesh of `nsing` points is used for $v_\perp \le$ `vbound`, and a coarser uniform mesh for $v_\perp >$ `vbound`. This provides higher resolution near the magnetic axis where gradients are typically steeper.

`ising = +1` Power-law spacing in $v_\perp$, $v_{\perp i} \propto [(i - \frac{1}{2})/(\texttt{nperp} - 1)]^p$ with $p = \texttt{p_grid}$ (default 2, quadratic), providing higher resolution near the axis $v_\perp = 0$. The `fd_stencil_2d` routine uses the Fornberg algorithm (`fornberg_weights`) to compute exact finite-difference weights for the local non-uniform spacing, so this mode is fully supported.

The cylindrical Jacobian is $J = 2\pi\, v_\perp$, so that integrals over velocity space are $\int\!\int f\, J\, \mathrm{d}v_\perp\, \mathrm{d}v_\parallel$.

The global 1D degree-of-freedom (DOF) index is

$$\mathrm{ix}(i, j) = (i - 1)\, N_\parallel + j, \quad i \in [1, N_\perp],\ j \in [1, N_\parallel], \tag{9}$$

giving $N_{\mathrm{dof}} = N_\perp \times N_\parallel$ total DOFs.

3.2 Finite-Difference Discretization

Spatial derivatives are discretized using the **7-point Fornberg stencil** (subroutine `fd_stencil_2d`), which is a fourth-order accurate scheme. The stencil uses 4 points in each direction: at the current node and its 3 neighbours. For a non-uniform $v_\perp$ grid, the Fornberg algorithm computes the exact derivative weights for the local grid spacing via subroutine `fornberg_weights`.

The stencil returns at most 49 entries per row (7 points in $v_\perp$ by 7 in $v_\parallel$), although boundary rows have fewer entries.

3.2.1 Boundary Conditions

Axis ($i = 1$, $v_\perp = v_{\perp,\min}$): Neumann condition $\partial f/\partial v_\perp = 0$, enforcing axial symmetry.

Outer boundary ($i = N_\perp$): Dirichlet condition $f = 0$.

Parallel boundaries ($j = 1$ and $j = N_\parallel$): Dirichlet condition $f = 0$.

3.3 Advection Stabilization at High $v_\perp$ (`i_upwind`)

The perpendicular drag coefficient B (collisional friction) grows with $v_\perp$ while the perpendicular diffusion D falls off on the supra-thermal tail, so the cell-Péclet number

$$\mathrm{Pe}_\perp = \frac{|B|\, \Delta v_\perp}{D} \tag{10}$$

increases steeply toward the outer boundary and can reach $\mathrm{Pe}_\perp \sim 40$ at $v_\perp = v_{\perp,\max}$ for high-energy (e.g. second-harmonic ICRF) scenarios. Beyond $\mathrm{Pe}_\perp \gtrsim 2$ a central discretization of the advection term is no longer dissipative; at the outer Dirichlet wall, where the Fornberg stencil is one-sided, the effective perpendicular diffusion of the discrete operator can even become *negative*. A modified-equation analysis of the Dirichlet-truncated wall row gives, in the uniform-spacing limit,

$$D_{\mathrm{eff}} \;=\; \frac{223}{360}\, D \;-\; \frac{1}{12}\, |B|\, \Delta v_\perp, \tag{11}$$

i.e. the discrete diffusion turns negative exactly when the wall-cell Péclet number exceeds $223/30 \approx 7.4$ (the deleted wall node biases the drag stencil downwind; high order shrinks the classic two-point downwind coefficient from $\frac{1}{2}$ to $\frac{1}{12}$ but cannot change its sign). The assembled operator $\mathcal{L}$ then acquires a small set of eigenvalues with positive real part, localized in a thin layer against the $v_\perp = v_{\perp,\max}$ boundary. Under Crank–Nicolson (`icn = -1`) these modes are amplified and the run eventually blows up, whereas the fully-implicit scheme damps them. The instability is governed by $v_{\perp,\max}$ (the wall velocity) and is essentially independent of the RF term and of the grid resolution.

Setting `i_upwind = -1` cures this at the discretization level: in every cell with $\mathrm{Pe}_\perp > 2$ the drag term $B\, \partial f/\partial v_\perp$ is evaluated with a first-order *upwind* one-sided difference (forward for $B > 0$,

backward for $B < 0$, i.e. the side that makes the semi-discrete eigenvalue dissipative), while the diffusion term keeps its central Fornberg weights. In the modified equation the upwind drag moment $+|B|\,\Delta v_\perp/2$ replaces $-|B|\,\Delta v_\perp/12$, so $D_{\text{eff}} > 0$ at *any* Péclet number; the $\text{Pe}_\perp > 2$ switching threshold is therefore conservative with respect to the ≈ 7.4 instability threshold above. The sparsity pattern is unchanged (the diffusion stencil already fills every same-column entry), so the phased-PARDISO machinery is untouched. This removes all positive-real-part modes with a bulk-physics change of at most a few percent, because the upwinding acts only on the near-empty tail. The default `i_upwind = 0` keeps the pure central scheme; running fully implicit (`icn = 0`) is an alternative stabilization. Note that first-order upwinding adds some numerical diffusion on the tail; a flux-limited or exponentially-fitted variant would reduce this if higher tail accuracy is needed.

3.4 Sparse Matrix Storage

All matrices are stored in **1-based Compressed Sparse Row (CSR)** format. For a matrix of size N_{dof}:

- `ia(1:`N_{dof}`+1)`: row pointer array.
- `ja(1:nnz)`: column indices.
- `aa(1:nnz)`: non-zero values.

Entries within each row are sorted by column index before passing to PARDISO. The maximum number of non-zeros per row is 49 (7×7 stencil), so `nnz_max` $= 49\,N_{\text{dof}}$.

3.5 Direct Solver: Intel MKL PARDISO

All linear systems are solved with **Intel MKL PARDISO** using matrix type `mtype = 11` (real non-symmetric general). The module `pardiso_solver` exposes four phased entry points that exploit PARDISO's factorization phases:

Entry point	PARDISO phases
`pardiso_solve_steady`	11 + 22 + 33 (full solve, then release)
`pardiso_solve_init`	11 + 22 (symbolic + numeric factorization)
`pardiso_solve_step`	33 only (back-substitution)
`pardiso_solve_finalize`	release internal memory

This phased approach is critical for performance in the time-dependent solver: when the LHS matrix is constant (linear problem), the factorization is done once and only the back-substitution is repeated at each time step.

4 Algorithms

4.1 Steady-State Solver

When `ntimes(1) = 0`, the code solves the steady-state FP equation

$$\mathcal{L}[f] = -S, \tag{12}$$

where $\mathcal{L}$ is the FP differential operator assembled from the coefficient arrays `all00`, ..., `all02`. The two-pass assembly (first count non-zeros, then fill values) builds the CSR matrix, which is then passed to `pardiso_solve_steady` for a direct solve (subroutine `FP_steady_state` in module `mod_linear`).

4.2 Time-Dependent Solver – Linear Problem (`timefp_7pt`)

Used when `ntimes(1) > 0` and `isc` $\neq -1$.

4.2.1 Time Discretization

A θ-method is used:

$$(I - \theta\,\Delta t\,\mathcal{L})\,f^{n+1} = (I + (1-\theta)\,\Delta t\,\mathcal{L})\,f^{n} + \Delta t\,S. \tag{13}$$

`icn`	θ	Scheme
-1	0.5	Crank–Nicolson
$+1$	0.75	Intermediate
else	1.0	Fully implicit

4.2.2 Algorithm

1. **Assemble** the FP operator $\mathcal{L}$ in CSR format (two-pass loop).
2. **Build LHS**: $M_{\text{lhs}} = I - \theta\,\Delta t\,\mathcal{L}$. The sparsity pattern is identical to $\mathcal{L}$ since the diagonal already exists.
3. **Factorize** M_{lhs} once (PARDISO phases 11+22).
4. **Time loop** for $n = 1, \ldots, N_{\text{times}}$:
   (a) Recompute the Coulomb logarithm and, for `isc` $\in \{1, 2, 3\}$, the Maxwellian self-collision background (at the fixed Stix, $T_{\text{eff}}(t)$, or $T_n(t)$ temperature, respectively) from the current distribution, then rebuild the FP operator (allowing a time-varying Coulomb logarithm and self-collision term).
   (b) Rebuild M_{lhs} with the updated operator.
   (c) Compute the RHS: $b = (I + (1-\theta)\,\Delta t\,\mathcal{L})\,f^n + \Delta t\,S$ via a hand-coded sparse matrix–vector product.
   (d) Enforce zero RHS on boundary DOFs.
   (e) Solve $M_{\text{lhs}}\,f^{n+1} = b$ (PARDISO phase 33 only).
   (f) Compute diagnostics (density, energy, power, momentum).
5. **Finalize** PARDISO.
6. If `isource = 0`: rescale $f \leftarrow f \cdot n_0 / \int f\,\mathrm{d}^3v$ to conserve total particle number.

4.3 Time-Dependent Solver – Nonlinear Self-Collisions (`timefp_7pt_nl`)

Used when `ntimes(1) > 0` and `isc = -1`. The LHS matrix now changes at every time step because the self-collision operator depends on f^n.

4.3.1 Phi-Distance Kernel Precomputation

Before entering the time loop, the phi-distance kernel $\Phi_{ij} = \phi(|\boldsymbol{v}_i - \boldsymbol{v}_j|)$ (Gauss–Legendre quadrature, $O(N_{\text{dof}}^2)$) is cached on disk in a per-case file `sum_phi-CASENAME.dat` (or `sum_phi.dat` when `casename` is empty). Since the kernel is grid-dependent, the file header stores a grid signature (`nperp`, `npar`, `vperp_min/max`, `vpar_min/max`). Caching is fully self-managing — no user flag is required:

Cache file state	Action
Missing	Compute kernel and save
Present, signature matches	Load kernel (skip the quadrature)
Present, signature differs or truncated	Recompute and overwrite

Because the signature is verified automatically, a stale kernel from a different velocity grid can never be silently reused.

4.3.2 Time-Stepping Scheme

The θ parameter has an additional option for the NL solver:

`icn`	θ	Comment
-1	0.5	Crank–Nicolson (may be unstable at high $v_\perp$; see Sec. 3.3)
$+1$	0.75	Intermediate
else	1.0	Fully implicit (recommended)

4.3.3 Algorithm

1. **Build sparsity pattern** of $\mathcal{L}$ using the linear coefficients only (pattern is fixed; NL SC uses the same stencil).
2. **Factorize symbolically** (PARDISO phase 11 only).
3. **Time loop** for $n = 1, \ldots, N_{\text{times}}$:
   (a) **Build NL self-collision term**: call `main_nlterm`(f^n) to compute the self-collision coefficient arrays `sc**` from the current distribution via the phi-distance kernel.
   (b) **Update Coulomb logarithm** and rebuild the linear FP coefficients.
   (c) **Assemble total operator**: $\mathcal{L} = \mathcal{L}_{\text{lin}} + \mathcal{L}_{\text{SC}}$.
   (d) **Rebuild LHS**: $M_{\text{lhs}} = I - \theta\,\Delta t\,\mathcal{L}$.
   (e) **Compute RHS**: $b = (I + (1-\theta)\,\Delta t\,\mathcal{L})\,f^n + \Delta t\,S$.
   (f) **Solve** (PARDISO phases 22+33 every step since M_{lhs} changes).
   (g) Compute diagnostics.
4. **Finalize** PARDISO.

Note on start-up: when starting from $f = 0$ with a beam source (i.e., `istart = 0`), the NL self-collision term is skipped for the first few steps until the beam density becomes significant enough for the Rosenbluth potentials to be numerically meaningful.

4.4 Multi-Phase Time-Stepping

The namelist parameters `ntimes` and `timestep` are each 3-element arrays, enabling up to three sequential phases with different time steps:

- Phase k runs `ntimes(k)` steps of duration `timestep(k)` seconds.
- Phases with `ntimes(k) = 0` are skipped.
- Phases 2 and 3 append to the output files (restart in time).

Old single-phase input files (`ntimes = N, timestep = dt`) remain backward-compatible: the scalar value fills element 1 and elements 2–3 default to zero.

4.5 Initial Condition for Time-Dependent Runs

The initial distribution f^0 is selected by `istart`:

`istart`	Initial condition
0	$f^0 = 0$ everywhere. Requires `isource = -1`.
1	Stix Maxwellian solution (default).
2	Steady-state solution of the linear solver without self-collisions.
3	Steady-state solution of the linear solver with Maxwellian SC background.

4.6 Steady-State Convergence Detection

When `i_ss_check = -1` the run stops early once the solution stops changing. A *single* test decides this, implemented in `mod_conv_diag`. It is evaluated once every $W =$ `n_ss_window` steps – not at every step – and measures rates of change over that window, exactly as the former scalar criterion did, but on the distribution function itself rather than on four integrated scalars. At each evaluation the current state is compared with the state stored at the previous evaluation, so only *one* snapshot of f is retained regardless of W, and the full-grid norms are formed once per window rather than once per step.

Phase-space weighting. f lives in cylindrical velocity space, so every norm and moment is integrated against $\mathrm{d}^3v = 2\pi v_\perp \,\mathrm{d}v_\perp \,\mathrm{d}v_\parallel$. Composite-trapezoidal weights on the (possibly non-uniform) grids are folded with the Jacobian into $W_\perp(i) = 2\pi v_{\perp,i}\, w_\perp(i)$ and $W_\parallel(j) = w_\parallel(j)$. Ignoring the Jacobian would over-weight the near-axis region, which holds almost no particles. As a check that both the Jacobian and the non-uniform weights are right, the code verifies the exact identity $\sum_i W_\perp(i) = \pi(v_{\perp,N}^2 - v_{\perp,1}^2)$ at startup.

Criteria. Writing f^{now} for the state at the current evaluation and f^{prev} for the one at the previous evaluation, and with $\mathrm{d}t_{\mathrm{win}} = t_{\mathrm{now}} - t_{\mathrm{prev}}$ the physical duration of the window, the test computes

$$\varepsilon = \frac{\|f^{\mathrm{now}} - f^{\mathrm{prev}}\|_2}{\mathrm{d}t_{\mathrm{win}}\, \|f^{\mathrm{now}}\|_2}, \qquad \varepsilon_{\mathrm{tail}} = \frac{\|f^{\mathrm{now}} - f^{\mathrm{prev}}\|_T}{\mathrm{d}t_{\mathrm{win}}\, \|f^{\mathrm{now}}\|_T}, \tag{14}$$

where $\|\cdot\|_2$ uses the weights above and $\|\cdot\|_T$ adds the tail weight $T = (1 + v^2/v_{\mathrm{th}}^2)^k$ $(k = 2)$. The Jacobian-weighted norm is dominated by the bulk, so $\varepsilon_{\mathrm{tail}}$ is reported *separately*: the interesting failure mode is ε small while $\varepsilon_{\mathrm{tail}}$ is not, i.e. a converged bulk over a still-evolving tail. The relative drift rates of the density, parallel flow and energy are computed over the same window; norm convergence is necessary but not sufficient, and a moment still drifting while ε is small points to a flux or boundary-condition error at $v_\perp = 0$ or at the domain edges rather than to a convergence failure.

With the default `i_conv_shape = 0` the run stops when *all five* conditions hold simultaneously: $\varepsilon/\nu_{\mathrm{ref}} <$ `ss_tol_eps`, $\varepsilon_{\mathrm{tail}}/\nu_{\mathrm{ref}} <$ `ss_tol_tail`, and each of the three moment drifts $<$ `ss_tol_moment`.

Amplitude vs shape (`i_conv_shape`). The rates above measure the change of f *including its amplitude*, which is the right thing when the run really does reach a steady state. In a *sourceless* run it is not: particles absorbed at the Dirichlet boundaries are never replaced, so once the VDF shape has settled f keeps decaying as $\phi(v)\, e^{-t/\tau}$ with the shape frozen. Then

$$\varepsilon \;\longrightarrow\; \frac{e^{\Delta t_{\mathrm{win}}/\tau} - 1}{\Delta t_{\mathrm{win}}} \;\approx\; \frac{1}{\tau}, \tag{15}$$

a floor that no tolerance below $1/\tau$ can ever clear, and ε, $\varepsilon_{\mathrm{tail}}$ and the density and energy drifts all collapse onto that same value (the tell-tale signature of a pure rescaling).

Setting `i_conv_shape = -1` selects the *shape* criterion: each field is normalised to unit norm before differencing,

$$\varepsilon_{\mathrm{shape}} = \frac{1}{\Delta t_{\mathrm{win}}} \left\| \frac{f^{\mathrm{now}}}{\|f^{\mathrm{now}}\|} - \frac{f^{\mathrm{prev}}}{\|f^{\mathrm{prev}}\|} \right\|, \tag{16}$$

so a uniform rescaling $f^{\mathrm{now}} = c\, f^{\mathrm{prev}}$ gives exactly zero. The moment tests then switch to the *intensive* moments — the parallel flow $u_\parallel$ and the mean energy per particle E/n, both unchanged by a uniform drain — and the density drift is dropped, being pure amplitude. All four rates are always computed and reported; `i_conv_shape` only selects which family the tolerances are applied to, and the log line is tagged `[amplitude]` or `[shape]` accordingly.

Verified 2026-07-30 on a 60×41 `isc = 2` RF case: the amplitude rates flattened at $\varepsilon \approx 2 \times 10^{-3}$ (the drain rate, with the density drift pinned at 1.73×10^{-3}) and the run never stopped, while $\varepsilon_{\mathrm{shape}}$ kept falling exponentially and reached 8.7×10^{-5} at $t = 12.5$ s, where the shape criterion stopped the run.

Units. ε has dimensions of inverse time, so it is only meaningful against a physical rate; it is divided by a reference rate ν_{ref} (the code passes $1/\tau_{ie}$) before comparison, and the tolerances are therefore dimensionless fractions of that rate. Because the window duration is taken from the simulation clock rather than a step count, the tolerances are *independent of the time step* and remain correct across phase boundaries where `timestep` changes.

Cost. One snapshot of f is retained ($n_\perp n_\parallel \times 8$ bytes, independent of W; the figure is printed at startup), and one full-grid pass is performed per window, i.e. $1/W$ of the cost of testing at every step.

Diagnostics are printed and written once per window – to `conv_eps_vs_time.txt` (time, ε, $\varepsilon_{\mathrm{tail}}$, $\varepsilon_{\mathrm{shape}}$, $\varepsilon_{\mathrm{tail,shape}}$) and `conv_diag_vs_time.csv` (those four rates, their ratios to ν_{ref}, the moments and all the drift rates). The first evaluation only lays down the reference state, so the first verdict comes at step $2W$.

4.7 Diagnostics and Output

At each time step the following quantities are written to disk:

- Density $n(t) = \int f\, J\, \mathrm{d}^3 v$.
- Total, perpendicular, and parallel kinetic energies.
- Effective temperature $T_{\mathrm{eff}} = (2T_\perp + T_\parallel)/3$, where $T_\perp = \frac{1}{2} m \langle v_\perp^2 \rangle$ and $T_\parallel = m \langle v_\parallel^2 \rangle$.
- Density-characteristic temperature T_n (the cold-bulk temperature from the $\ln f$-vs-v^2 log-slope fit; it is the background used by `isc = 3` but is written as a diagnostic for *all* runs).
- Near-axis and whole-grid minima of f (a negative-f monitor: small negative undershoots near the axis can corrupt low-velocity moments such as T_n).
- Perpendicular anisotropy factor $= 100\, T_\perp/(T_\perp + T_\parallel)$ (%), ranging from 0% (all-parallel) to 50% (Maxwellian) to 100% (all-perpendicular).
- Power densities (collisional by species, RF, beam source, losses, self-collision total), and their perpendicular/parallel split for self-collisions.
- Momentum transfer rates (perpendicular and parallel) for each operator.

- Coulomb logarithm(s) vs. time.

At the end of a run, balance diagnostics (power, density, and optionally momentum) are printed to standard output.

5 Namelist Reference

The code reads its input from standard input (`stdin`) as a Fortran `NAMELIST` group named `INPUT`. A typical invocation is:

```
FP2D_QLRF_NL.exe < jet_beam_case1.dat
```

The namelist file may omit any variable that has a default value; the code will use the default.

The parameters are grouped thematically below.

5.1 General and Output Control

Parameter	Default	Description
`casename`	`"`	Character string (max. 64 chars) appended to every output filename and plot title, e.g. `'JET-beam7-TD0-NLSC'`. Leave empty for unnamed runs.
`iplot_pow`	-1	Controls writing of power-vs-time output files. -1: write all power files. 0: skip (useful when only energy and density diagnostics are needed).
`iplot_mom`	0	Controls writing of momentum-transfer-rate files. -1: write all momentum files. 0: skip (default; momentum files are large).
`n_snap`	0	VDF snapshots during a time-dependent run. 0: none. $N > 0$: write f every N steps to `vdf_snap_<step>.txt` (Section 6.2). Both time solvers.
`notxt`	0	Controls how many `.txt` output files are written (the `.dat` restart/kernel files are never affected). 0: write all `.txt` outputs (normal). 1: suppress all `.txt` outputs. 2: write only the minimal test/diagnostic set, namely `Ekin_perp`, `Ekin_perp_at_vpar0`, `fout_at_vpar0`, `Teff_vs_time`, and `anisotropy_vs_time`. Suppressed files are routed to the OS null device, so the run is otherwise unchanged (all stdout diagnostics still print).

5.2 Velocity-Space Grid

Parameter	Default	Description
`nperp`	–	Number of grid points in $v_\perp$ (perpendicular direction).
`npar`	–	Number of grid points in $v_\parallel$ (parallel direction).

Parameter	Default	Description
`vperp_min`	–	Minimum perpendicular velocity (m/s). Should be 0 or very small.
`vperp_max`	–	Maximum perpendicular velocity (m/s). Should be at least 3–5× the beam or thermal velocity.
`vpar_min`	–	Minimum parallel velocity (m/s). Must be negative.
`vpar_max`	–	Maximum parallel velocity (m/s). Must be positive and equal to $\lvert$`vpar_min`$\rvert$ for symmetric grids.
`ising`	0	Grid spacing mode in $v_\perp$. 0: uniform. -1: two-domain (fine for $v_\perp \leq$ `vbound`, coarser above). $+1$: power-law spacing (quadratic by default, see `p_grid`), higher resolution near $v_\perp = 0$.
`p_grid`	2	Exponent of the power-law grid when `ising = +1`: $v_{\perp i} =$ `vperp_min` $+$ `vperp_max` $\left[(i - \frac{1}{2})/(\texttt{nperp} - 1)\right]^p$. $p = 2$ is the original quadratic grid (reproduced to the bit); a larger p moves points from the tail into the thermal bulk and coarsens the spacing at the $v_\perp$ wall, which raises the wall Péclet number. Ignored for `ising` $\neq +1$.
`nsing`	–	Number of fine grid points in the inner domain when `ising = -1`. Must satisfy $1 \leq$ `nsing` $<$ `nperp`.
`vbound`	–	Boundary between fine and coarse regions in $v_\perp$ when `ising = -1` (m/s).

5.3 Plasma Parameters

Parameter	Default	Description
`nbulk`	–	Number of background species (including electrons). Species 1 is always electrons. Up to 10 species are supported.
`t(1:nbulk)`	–	Temperatures of background species (eV). `t(1)`: electron temperature T_e. `t(2)`: temperature of first ion species, etc.
`aa`	–	Mass number of the minority (test) ion species.
`za`	–	Charge number (atomic number) of the minority ion species.
`ab(1:9)`	–	Mass numbers of background ion species 2, 3, … (species 1 is always electrons with m_e/m_p).
`zb(1:9)`	–	Charge numbers of background ion species 2, 3, …
`ne`	–	Electron density (m^{-3}).
`xpart`	–	Concentration of minority ion species relative to n_e, i.e. $n_a =$ `xpart` $\times\, n_e$.
`xb(1:9)`	–	Concentrations of background ion species 2, 3, … relative to n_e. Quasi-neutrality is assumed: $\sum_b Z_b\, n_b = n_e$.

5.4 Beam Source

Parameter	Default	Description
`isource`	–	Beam source flag. -1: include NBI beam source and particle losses. 0: no source; distribution is sourceless (total particles conserved).
`beam_ekin`	–	Beam injection energy (keV).
`beam_angle_deg`	–	Injection angle (degrees) measured from the magnetic field direction. 0°: purely parallel injection. 90°: purely perpendicular injection.
`beam_dvperp`	–	Width $\sigma_\perp$ of the Gaussian approximation to the delta function in $v_\perp$ (m/s). A value of $\sim$ 5–10% of $v_\perp^{\mathrm{beam}}$ is typical; must be resolved by the grid.
`beam_dvpar`	–	Width $\sigma_\parallel$ of the Gaussian in $v_\parallel$ (m/s).
`taus`	–	Slowing-down time τ_s (s). Sets the source amplitude $S_0 = n_a/\tau_s$ and the loss rate.

5.5 RF Heating

Parameter	Default	Description
`irf`	–	RF flag. -1: include quasi-linear RF operator. 0 or else: no RF.
`frek`	–	Wave frequency f (Hz).
`b0`	–	Static magnetic field B_0 (T).
`nharm`	–	Resonance harmonic number n (e.g. 1 for fundamental, 2 for second harmonic).
`kpar`	–	Parallel wave vector $k_\parallel$ (rad/m).
`kperp`	–	Perpendicular wave vector $k_\perp$ (complex, rad/m).
`eplus`	–	Left-hand polarisation amplitude E_+ (complex, V/m).
`emin`	–	Right-hand polarisation amplitude E_- (complex, V/m).
`delta_RF`	–	Width of the Gaussian broadening of the resonance in $v_\parallel$ (m/s). Must be wide enough to resolve the resonance on the grid.

5.6 Time Integration and Solver Control

Parameter	Default	Description
`ntimes(1:3)`	`0,0,0`	Number of time steps per phase. `ntimes(1) = 0`: steady-state run. `ntimes(1) > 0`: time-dependent run. Up to three phases with different step sizes are supported; phases with `ntimes(k) = 0` are skipped. Old-format scalar values (`ntimes = N`) are backward-compatible and fill `ntimes(1)`.
`timestep(1:3)`	`0,0,0`	Time step size per phase (s). `timestep(k)` is used for phase k. Backward-compatible with scalar input.

Parameter	Default	Description
`icn`	–	Time-differencing scheme. -1: Crank–Nicolson ($\theta = 0.5$). $+1$: intermediate scheme ($\theta = 0.75$). else: fully implicit ($\theta = 1.0$).
`iold`	–	Initial condition source. 0: fresh start (use `istart` to select the initial condition). -1: restart from a previous run by reading `xout.dat`.
`istart`	1	Initial distribution for fresh time-dependent runs (`iold = 0`). 0: $f^0 = 0$ (beam-driven only; requires `isource = -1`). 1: Stix Maxwellian. 2: Steady-state solution without self-collisions. 3: Steady-state solution with Maxwellian self-collisions.
`isc`	–	Self-collision treatment. 0: no self-collisions (linear FP equation). 1: Maxwellian background at fixed Stix temperature. 2: Maxwellian background at evolving effective temperature $T_{\mathrm{eff}}(t)$ (requires TD run). 3: Maxwellian background at the density-characteristic temperature $T_n(t)$ (requires TD run; see `core_frac`). -1: fully nonlinear self-collision operator (requires TD run).
`core_frac`	$3.8{\times}10^{-3}$	Core fraction defining the T_n log-slope fit for `isc = 3`: only grid cells with $f >$ `core_frac` $\cdot \max f$ enter the $\ln f$-vs-v^2 fit. Smaller values weight the cold bulk more heavily. Ignored unless `isc = 3`; the appropriate value is scenario-dependent.
`i_upwind`	0	Advection stabilization of the perpendicular drag term $B\,\partial f/\partial v_\perp$ (Section 3.3). 0: central Fornberg weights (default). -1: Péclet-hybrid upwinding — in cells where the cell-Péclet number $\lvert B\rvert\,\Delta v_\perp/D > 2$ the drag term uses a one-sided (first-order) upwind difference, restoring dissipativity of the advection-dominated outer boundary layer and removing the Crank–Nicolson high-$v_\perp$ instability. 1: Péclet-hybrid *power-law* scheme (Patankar 1980) — in the same advection-dominated cells the combined drag+diffusion operator is discretised with a 3-point control-volume power-law stencil, which removes the instability with roughly half the tail numerical diffusion of the first-order upwind (-1). Both leave the high-order 7-point Fornberg stencil in the smooth bulk.
`i_lu_reuse`	-1	Linear solves of the time steps. -1: reuse the LU factors of the last factorisation as the preconditioner of a CGS iteration, and refactorise only when its result is rejected (Section 8.1); 2.7–3.4× faster runs. 0: refactorise the matrix at every step (the former behaviour).
`lu_reuse_tol`	10^{-10}	Residual bound for `i_lu_reuse = -1`: a reused step is accepted if $\lVert b - Ax\rVert_\infty / \lVert b\rVert_\infty \le \max($`lu_reuse_tol`$, 10\,r_{\mathrm{direct}})$, with r_{direct} the residual of the latest direct solve. Values below $\sim 10^{-11}$ make every step refactorise.

Parameter	Default	Description
`n_threads`	0	Number of threads for MKL (PARDISO) and the OpenMP loops (Release build). 0: `MKL_NUM_THREADS` / `OMP_NUM_THREADS` if set, otherwise 16. $n > 0$: n threads for both, overriding the environment. 1 is the sequential code: results identical to the sequential (Debug) build and reproducible from run to run; with more threads they differ in the last digits. No effect in the Debug build (Section 8.1).

5.7 Steady-State Convergence Detection

Parameter	Default	Description
`i_ss_check`	0	Enable automatic convergence detection (Section 4.6). 0: disabled (run for the full `ntimes` steps). -1: stop early when all five criteria are met; also writes `conv_eps_vs_time.txt` and `conv_diag_vs_time.csv`.
`n_ss_window`	50	Window width W (number of time steps). The convergence test is evaluated once every W steps, comparing the current state with the one stored at the previous evaluation and dividing by the physical duration the window spans. A wider window is less sensitive to short-term fluctuation and proportionally cheaper, but coarsens the time resolution of the diagnostic and delays the first verdict (which comes at step $2W$). $W = 1$ evaluates every step.
`ss_tol_eps`	10^{-3}	Tolerance on $\varepsilon/\nu_{\mathrm{ref}}$, the Jacobian-weighted bulk convergence rate, Eq. (14). Dimensionless (a fraction of the reference rate) and independent of `timestep`.
`ss_tol_tail`	10^{-2}	Tolerance on $\varepsilon_{\mathrm{tail}}/\nu_{\mathrm{ref}}$, the tail-weighted rate. Looser than `ss_tol_eps` by default because the tail is the noisier, least-populated part of the distribution; tighten it when the physics of interest lives in the tail.
`ss_tol_moment`	10^{-3}	Tolerance on the relative moment drift rates over the window. Which moments are tested depends on `i_conv_shape`: with 0 the density, parallel flow and energy; with -1 the intensive moments only, i.e. the parallel flow and the mean energy per particle E/n.

Parameter	Default	Description
`i_conv_shape`	0	Selects *which family of rates the tolerances are applied to* (Section 4.6). Both families are always computed and written out; this flag changes only which ones are tested. 0 — *amplitude*. The criterion uses ε and $\varepsilon_{\mathrm{tail}}$, Eq. (14), together with the drift rates of n, $u_\parallel$ and E. Five conditions in all. This is the correct choice when the run reaches a genuine steady state, e.g. when a beam source balances the boundary losses. -1 — *shape*. f is normalised to unit norm before differencing, Eq. (16), so the criterion uses $\varepsilon_{\mathrm{shape}}$ and $\varepsilon_{\mathrm{tail,shape}}$ together with the drift rates of the *intensive* moments $u_\parallel$ and E/n; the density drift is dropped, being pure amplitude. Four conditions in all. Required for a *sourceless* run: particles absorbed at the Dirichlet boundaries are never replaced, so f decays as a frozen shape with falling amplitude and the amplitude rates floor at the drain rate, which no tolerance below that rate can ever clear.

Convergence requires *all* the selected conditions to be satisfied simultaneously at one evaluation. If a run never stops, compare ε with $\varepsilon_{\mathrm{shape}}$ in `conv_eps_vs_time.txt`: if the former has flattened while the latter is still falling, the run is draining and needs `i_conv_shape = -1`.

6 Output Files

All output files are written to the working directory. When `casename` is non-empty, the string `-casename` is inserted before the file extension.

6.1 Final Distribution

`fout.txt` Final VDF as a list of $(v_\perp, v_\parallel, f)$ triples.

`xout.dat` Restart file: one header line, `time nstep` (final time and total number of steps, including those of any run it was itself restarted from), followed by the packed 1D solution vector. Used with `iold = -1`. Older files whose header holds only the time are still accepted; the step count then starts from 0.

`fstix.dat` Stix Maxwellian reference distribution.

`Ekin.txt`, `Ekin_perp.txt`, `Ekin_par.txt`, `Ekin_perp_at_vpar0.txt` Kinetic-energy density of the final f (total, perpendicular and parallel parts), and its cut at $v_\parallel = 0$.

`fsc_maxw.txt`, `fsc_maxw_at_vpar0.txt` Maxwellian background of the self-collision operator, at T_{eff} (`isc = 2`) or T_n (`isc = 3`).

`beam.txt` NBI source $S(v_\perp, v_\parallel)$ (`isource = -1`).

6.2 VDF Snapshots

With `n_snap = N > 0`, f is written every N steps to `vdf_snap_<step>.txt`, in the same layout as `fout.txt` with a `# time =` header line. `<step>` is the *total* step count read from the `xout.dat`

header, so a restart continues the previous run's numbering and cadence instead of overwriting its snapshots. Snapshots hold the raw solution (the end-of-run renormalisation of sourceless runs is not applied) and are not restart files. `fp2d_plot.py -movie` turns them into an animation (Section 8.3).

6.3 1D Cuts of the VDF

`fout_at_vpar0.txt`, `fout_at_vperp0.txt`, `fout_at_vperpmax.txt`, `fout_at_vparmax.txt`: values of f along the four boundary slices of the domain.

6.4 Time Traces

Written at every time step during the simulation (except the convergence files, which are written once per `n_ss_window`-step window):

`density_vs_time.txt` Columns: time (s), $n(t)$ (m^{-3}).

`energy_vs_time.txt` Columns: time, E_{tot} (keV), $E_{\perp}$ (keV).

`anisotropy_vs_time.txt` Columns: time, perpendicular anisotropy (%).

`Teff_vs_time.txt` Columns: time, T_{eff} (keV).

`Tn_vs_time.txt` Columns: time, T_n (keV) – density-characteristic (cold-bulk) temperature; written for all runs.

`conv_eps_vs_time.txt` Columns: time, ε, $\varepsilon_{\mathrm{tail}}$, $\varepsilon_{\mathrm{shape}}$, $\varepsilon_{\mathrm{tail,shape}}$ (s^{-1}) – the convergence rates, Eqs. (14) and (16). Written when `i_ss_check = -1`, one row per `n_ss_window`-step window.

`conv_diag_vs_time.csv` Full convergence history: the four rates above, their ratios to ν_{ref}, the moments n, $u_{\parallel}$, E, E/n, all four drift rates and the unweighted axis-row rate.

`power_coll_tot_vs_time.txt` Total collisional power density ($\mathrm{MW/m}^3$).

`power_coll_e_vs_time.txt` Electron collisional power density.

`power_coll_ion`N`_vs_time.txt`** Ion N collisional power density.

`power_coll_self_vs_time.txt` Self-collision power density (4 columns: time, total, $P_{\mathrm{SC}}^{\perp}$, $P_{\mathrm{SC}}^{\parallel}$).

`power_NBI_vs_time.txt` Beam injection and particle loss power.

`power_RF_vs_time.txt` RF power density.

`coulomb_log_vs_time.txt` Coulomb logarithm for background ion species.

`coulomb_log_self_vs_time.txt` Self-collision Coulomb logarithm.

`density_terms_vs_time.txt` Particle rate $\mathrm{d}n/\mathrm{d}t$ carried by each term of the operator ($\mathrm{m}^{-3}\,\mathrm{s}^{-1}$): time, total, collisions with each background species (electrons first), self-collisions, RF, source, losses. Each term is the velocity integral of its own part of $\mathcal{L}f$; every term is a divergence, so a non-zero value means particles leave through the Dirichlet walls or the term is not conservative in its discrete form. Written when `iplot_pow = -1`.

`momentum_coll_*_vs_time.txt` Momentum transfer rates (2 columns per file: $\perp$ and $\parallel$ components), written when `iplot_mom = -1`. The RF, NBI and self-collision rates go to `momentum_RF`, `momentum_NBI` and `momentum_coll_self`.

`tau_coll_vs_time.txt` Columns: time, effective τ_{ii}, τ_{ie} (s).

tau_rf_vs_time.txt Columns: time, RF tail-formation time (s); written when `irf = -1`.

6.5 Other Files

phi_kern-<casename>.dat Cache of the compressed ϕ-kernel of the nonlinear self-collision operator (`isc = -1`); reused by a later run on the same grid, rebuilt otherwise. Not a diagnostic; may be deleted.

RF_dirac.txt Parallel resonance function of the RF operator (`irf = -1`, `notxt = 0`). Not routed through `casename`: every run overwrites it.

6.6 End-of-Run Balance Diagnostics (stdout)

Power, particle density and momentum ($\perp$ and $\parallel$) balances are printed to standard output at the end of every run. Values are given in MW/m^3 for power, N/m^3 for momentum, and s^{-1} for particle rates. (The former switch `idiag`, which selected them, has been removed; a namelist that still sets it is rejected at read time.)

7 Example Namelist

The following example illustrates a time-dependent D-beam simulation in a D plasma with nonlinear self-collisions, using a two-phase time-stepping strategy: 1000 coarse steps followed by 1000 finer steps. It lists *every* `&INPUT` parameter; the RF block is included for completeness but is only read when `irf=-1`, and the diagnostic/advanced flags are shown at their default (off) values.

```
&INPUT
  casename = 'JET-beam7-TD0-NLSC',

  ! Grid
  nperp = 151, npar = 151,
  vperp_min = 0.d0, vperp_max = 5.d6,
  vpar_min = -4.d6, vpar_max = 4.d6,
  ising = 1, nsing = 25, vbound = 0.07d6,
  p_grid = 2.d0,

  ! Plasma: electrons + D ions
  nbulk = 2,
  t = 15.d3, 10.d3, 8*0.d0,
  aa = 2.d0, za = 1.d0,
  ab = 2.d0, 8*0.d0,
  zb = 1.d0, 8*0.d0,
  ne = 5.0d19,
  xpart = 0.05d0,
  xb = 0.50d0, 8*0.d0,

  ! D-NBI beam at 120 keV, 90 degrees
  isource = -1,
  beam_ekin = 120.d0,
  beam_angle_deg = 90.d0,
  beam_dvperp = 0.12d6,
  beam_dvpar = 0.20d6,
  taus = 300.d-3,

  ! RF quasi-linear operator (b0..delta_RF read only when irf = -1)
  irf = 0,
```

```
  b0 = 3.45d0,
  frek = 53.d6,
  nharm = 1,
  kpar = 2.d0,
  kperp = (30.d0, 1.d0),
  eplus = (1.d3, 3.25d3),
  emin = (3.d3, 3.75d3),
  delta_RF = 0.5d6,

  ! Time integration: two phases
  icn = 1,
  ntimes = 1000, 1000, 0,
  timestep = 1.d-3, 10.d-3, 0.d0,
  iold = 0,
  istart = 0,
  isc = -1,
  core_frac = 3.8d-3,

  ! Steady-state convergence detection
  i_ss_check = -1,
  n_ss_window = 50,
  ss_tol_eps = 1.d-3,
  ss_tol_tail = 1.d-2,
  ss_tol_moment = 1.d-3,
  i_conv_shape = 0,

  ! Output
  iplot_pow = -1,
  iplot_mom = -1,
  notxt = 0,
  n_snap = 0,

  ! Advanced numerics (defaults shown)
  i_upwind = 0,
  i_lu_reuse = -1,
  lu_reuse_tol = 1.d-10,
  n_threads = 0, ! 0: 16, or MKL_NUM_THREADS; n > 0: n threads
/
```

8 Compilation and Execution

8.1 Build

Open the Visual Studio solution `FP2D_QLRF_NL/FP2D_QLRF_NL.sln` and build the `Release|x64` configuration. The compiler is Intel `ifx` with Intel MKL; the Release build links the *threaded* MKL, so the sparse solver PARDISO runs in parallel, and is compiled with OpenMP, which parallelises the code's own loops over the grid: the matrix assembly, the self-collision potentials and the per-step diagnostics (the Debug build is sequential). The executable is placed at `FP2D_QLRF_NL/x64/Release/FP2D_QLRF_NL.exe`.

Number of threads. The code uses 16 threads by default (fewer on a machine with fewer cores) and prints the number at start. To change it, set `n_threads` in the namelist: `n_threads = 4` uses four threads for both MKL (PARDISO) and the OpenMP loops, and overrides the environment. With `n_threads = 0` (the default) the environment variable `MKL_NUM_THREADS` (or `OMP_NUM_THREADS`) is used if set before running, e.g. `set MKL_NUM_THREADS=4` in a Windows

command prompt, and 16 otherwise. There is no separate sequential solver: `n_threads = 1` is the sequential code, with results identical to the sequential (Debug) build and reproducible from run to run. The thread count is chosen at run time; only the choice between threaded and sequential MKL is fixed at compilation (Release and Debug respectively). MKL's own default, one thread per physical core, is slower than 8–16 threads on a many-core machine, which is why the code sets its own. The OpenMP loops use the same number unless `OMP_NUM_THREADS` is set. On a 150×150 grid with nonlinear self-collisions and RF, 100 steps took 29.4 s with the original sequential code and take 3.0 s now on 16 threads (with LU reuse, below); a full 900-step run takes 21 s. The gain saturates at 8–16 threads. The OpenMP loops do not change the results; with one thread the results are identical to the sequential build, and with more the parallel factorization in MKL makes them differ in the last digits (up to $\sim 10^{-5}$ of $\max f$ after 100 steps), because it eliminates in a different order. When running several cases at once, give each only a few threads.

LU reuse. The matrix $I - \theta\,\Delta t\,L$ changes from step to step, but slowly. With `i_lu_reuse = -1` (the default) the code therefore factorises it only occasionally: at the other steps it solves by a few CGS iterations preconditioned with the last LU factors (PARDISO phase 23), checks the true residual of the result, and refactorises only if that residual exceeds $\max(\texttt{lu_reuse_tol},\, 10\, r_{\mathrm{direct}})$. The end of the run prints how many factorisations were needed and the largest residuals, e.g.

```
PARDISO: 900 steps, 2 factorisations; LU reused on 898 steps
        (2.8 CGS iterations on average, 1 rejected)
```

On the ITER $N = 2$ reference case this cut a full run from 142 s to 52 s and on a JET beam fill-up from 96 s to 28 s (16 threads); with one thread, 100 steps went from 28.7 s to 8.3 s. The results agree with refactorising every step to $\sim 10^{-8}$ of $\max f$ on one thread, and within the threading noise on 16. The finite-difference weights, which depend only on the grid, are also computed once per run instead of at every step.

8.2 Running

The program reads from `stdin`:

```
FP2D_QLRF_NL.exe < inputs/jet_beam_case1.dat
```

Output files are written to the working directory.

8.3 Plotting

Use the companion Python script `fp2d_plot.py`:

```
python fp2d_plot.py run FP2D_QLRF_NL.exe inputs/jet_beam_case1.dat --show
python fp2d_plot.py plot x64/Release --casename JET-beam7-TD0-NLSC --save plots/
python fp2d_plot.py compare x64/Release --cases JET-beam7-TD0-Lin JET-beam7-TD0-NLSC
python fp2d_plot.py plot x64/Release --cases JET-beam7-TD0-NLSC --movie
```

The last line builds an animation of f, the kinetic-energy density and its perpendicular and parallel parts (a 2×2 array of panels) from the `vdf_snap` files (requires `n_snap > 0`); `-movie-scale frame` rescales each frame to its own maximum, and `-movie-format` selects AVI (default), MP4 or GIF. See `fp2d_plot-readme.txt` for the full documentation of the plotting tool.